

מבוא לתכנות מערכות – מבחן מועד ב', סתיו תשע"ו

מרצה: ד"ר איתי שרון

משך המבחן: 180 דקות

חומר עזר מותר: 2 דפים המכילים תאור פקודות ב-C ו-bash. כל חומר עזר אחר אסור.

המבחן כולל 5 שאלות, יש לענות על כולן. בשאלות הדורשות נימוק הקפידו לספק אותו.

יש לענות על טופס הבחינה במקומות המיועדים לכך. ניתן להשתמש במחברת העזר אולם יש להגיש רק את טופס הבחינה.

העמוד האחרון במבחן כולל חומר שיכול (אך לא חייב!) לעזור לכם.

בהצלחה!

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 1 (20 נקודות)

א. נתונה הפונקציה הבאה. תארו במילים מה היא עושה.

```
bool creepy(const char* s)
{
    const char* t = s+strlen(s)-1;
    while(t > s) {
        if(*(s++) != *(t--))
            return false;
    }
    return true;
}
```

הפונקציה בודקת האם המחרוזת שהועברה ע"י s היא פלינדרום.

ב. ממשו גרסא של הפקודה wc שמקבלת שם של קובץ ומדפיסה את מספר התווים, המילים והשורות בפורמט כרצונכם ל-stdout. הניחו ש:

- שורה מסתיימת בתו ירידת שורה ('\n')
- מילים מופרדות ע"י תו white space (רווח, טאב, ירידת שורה) אחד בדיוק, ו-white spaces מופיעים רק בין מילים (כלומר: מספרם שווה למספר המילים). ניתן לבדוק האם תו הוא white space ע"י קריאה לפונקציה isspace(char c) שמחזירה true אם התו הוא תו רווח או false אחרת.
- אין צורך לבדוק שהקובץ נפתח בצורה נכונה.

```
void my_wc(const char* file name) {
    FILE* fp = fopen(file name, "r");
    int nlines=0, nwords=0, nchars=0;
    char* buf = NULL;
    size_t bufsize = 0;
    while(getline(&buf, &bufsize, fp) > 0) {
        const char* p;
        nlines++;
        nchars += strlen(buf);
        for(p=buf; *p!='\0'; p++)
            nwords+=isspace(*p);
    }
    fclose(fp);
    printf("\t%d\t%d\t%d\t%s\n", nlines, nwords, nchars, file name);
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 2 (35 נקודות)

קבוצה (Set) הוא מבנה נתונים שבו כל איבר יכול להופיע בדיוק פעם אחת. בכתה דנו במימוש של קבוצה בעזרת רשימה מקושרת ומערך של ביטים וראינו מימוש של קבוצה בעזרת רשימה מקושרת. שאלה זו מתייחסת למימוש של קבוצה בעזרת מערך של ביטים. מימוש זה אינו מאפשר לשמור את האלמנטים עצמם אלא רק את האינדקס שלהם שניתן על ידי המשתמש. כאשר המשתמש מעוניין להתייחס לאלמנט מסויים (למשל בחיפוש, הכנסה או הוצאה) הוא מעביר את האינדקס של האלמנט ולא את האלמנט עצמו. לדוגמא: ניתן לבדוק "האם איבר 5 נמצא" או "להכניס את איבר מספר 2" או "להוציא את איבר מספר 31".

המימוש בשאלה זו צריך להיות לא מוגבל מבחינת מספר האלמנטים אותם ניתן להכניס.

א. כתבו header file עבור ADT של קבוצה במימוש של ביטים. הקובץ צריך להכיל הצהרות על הפונקציות הבאות:

- פונקציית יצירת קבוצה
- פונקציית הריסת קבוצה
- הוספת איבר
- הסרת איבר
- חיפוש איבר (הפונקציה מחזירה תשובה לגבי האם האיבר קיים או לא קיים)
- החזרת מספר האיברים בקבוצה
- איחוד של שתי קבוצות

```
#ifndef SET H
```

```
#define SET H
```

```
typedef struct Set* Set
```

```
Set SetCreate();
```

```
void SetAdd(Set s, int element);
```

```
void SetRemove(Set s, int element);
```

```
bool SetExists(Set s, int element);
```

```
int SetNumElements(Set s);
```

```
/* For the following it is also ok to add s2's elements to s1's */
```

```
Set SetUnion(Set s1, Set s2);
```

```
#endif
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. כתבו את ה-structs (אחד או יותר) שבהם תעשו שימוש עבור מימוש הקבוצה שלכם. ממשו את פונקציית יצירת הקבוצה.

```
struct Set {
```

```
    unsigned char* bit elements;
```

```
    unsigned int bit elements size;
```

```
};
```

```
Set SetCreate()
```

```
{
```

```
    Set s = (Set)malloc(sizeof(struct Set));
```

```
    if(s == NULL) {
```

```
        printf("Failed to allocate memory in Set SetCreate()\n\n");
```

```
        exit(-1);
```

```
    }
```

```
    s->bit elements = NULL;
```

```
    s->bit elements size = 0;
```

```
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ג. ממשו את הפונקציה המחזירה את מספר האיברים בקבוצה.

```
int SetNumElements(Set s)
```

```
{
```

```
    int i, j, n=0;
```

```
    for(i=0; i<s->bit elements size; i++) {
```

```
        for(j=0; j<8; j++) {
```

```
            n += ((*s->bit elements+i) & (1 << j)) != 0;
```

```
        }
```

```
    }
```

```
    return n;
```

```
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 3 (15 נקודות)

בשאלה זו אתם מתבקשים לממש אלגוריתם מיון גרי פשוט.

א. ממשו את הפונקציה `max_index` אשר מקבלת מערך של `void*`, את אורך המערך ופרמטרים נוספים לבחירתכם ומחזירה את האינדקס של האיבר הגדול ביותר.

```
int max_index(void* arr[], int size, bool (*is larger)(void*, void*)) {  
    int res = 0, i;  
    for(i=1; i<size; i++) {  
        if(is larger(arr[res], arr[i])) {  
            res = i;  
        }  
    }  
    return res;  
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. ממשו פונקציה אשר מקבלת מערך של מצביעים מטיפוס `void*`, את אורך המערך ופרמטרים נוספים לבחירתכם וממיינת את המערך. ניתן להשתמש באלגוריתם לבחירתכם או להשתמש באלגוריתם הבא:

1. קלט: מערך (`array`) והאורך שלו (`n`)
2. עבור כל אינדקס `i` מ-0 עד `n-1`:
3. מצא את האינדקס `j` של האיבר המקסימלי מתוך `array[i..(n-1)]`
4. החלף בין `array[i]` ל-`array[j]`

ניתן (ואף מומלץ!) להשתמש בפונקציה מהסעיף הראשון גם אם לא ממשתם אותה. ניתן (ואף מומלץ...) להשתמש בפונקציה הבאה:

```
void swap(void** a, void** b)
{
    void* temp = *a;
    *a = *b;
    *b = temp;
}
```

```
void my sort(void* arr[], int size, bool (*is larger)(void*, void*)) {
    int i, j;
    for(i=0; i<size; i++) {
        j = max index(arr+i, size, is larger);
        if(i != j) {
            swap(&(arr[i]), &(arr[j]));
        }
    }
}
```

שאלה 4 (20 נקודות)

שאלה זו עוסקת בעצי חיפוש בינאריים.

להלן ממשק חלקי של עץ חיפוש בינארי הדומה לזה שראינו בכתה.

```
struct BNode {
    struct BNode *left, *right;
    int data;
};
```

```
typedef struct BNode *BNode;
```

```
BNode BSTAddNode(BNode, int);
BNode BSTRemoveNode(BNode, int);
bool BSTSearchNode(BNode, int);
void BSTinorder(BNode);
```

א. תארו כיצד עובדת הפונקציה **BSTAddNode** (כפי שלמדנו בכתה או מימוש שלכם). בנוסף, ציירו את עץ החיפוש שיתקבל כתוצאה מהרצת הקוד הבא.

```
int main()
{
    int array[10] = {17, 11, 12, 24, 21, 35, 49, 5, 18, 22};
    int i;

    BNode root = NULL;
    for(i=0; i<10; i++)
        BSTAddNode(root, array[i]);
    Return 0;
}
```

הפונקציה תשווה בכל צומת את האיבר אותו רוצים להכניס לאיבר שבצומת הנוכחי ואז תפנה ימינה אם האיבר להכנסה

גדול מהאיבר הנוכחי או שמאלה אם הוא קטן או שווה לנוכחי. כשתקרא הפונקציה עבור NULL היא תייצר צומת חדש

עם האיבר להכנסה ותחזיר את כתובת הצומת כדי שהקריאה הקודמת תחבר את הצומת החדש כנדרש לעץ

(בן ימני או שמאלי). 17

11 24

5 12 21 35

18 22 49

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. כתבו פונקציה אשר בודקת האם עץ חיפוש בינארי מכיל כפילויות. במידה והעץ מכיל ערכים שמופיעים יותר מפעם אחת הפונקציה תחזיר true, אחרת הפונקציה תחזיר false. לצורך פתרון סעיף זה אין להשתמש בפונקציות המוגדרות ב-header. רמז: חשבו על הצורה שבה מודפסים הערכים שבעץ חיפוש בינארי בצורה ממויינת.

```
bool AreDuplicatesPresentInBST (BNode root) {  
  
    static int n=0;          /* Number of element to be looked at */  
  
    static int last val = 0; /* Last elements we've looked at */  
  
    if(!root)  
  
        return false;  
  
    if(AreDuplicatesPresentInBST(root->left) == true)  
  
        return true;  
  
    if((n > 0) && (root->value == last val))  
  
        return true;  
  
    n++;  
  
    last val = root->value;  
  
    return AreDuplicatesPresentInBST(root->right)  
  
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 5 (10 נקודות)

כתבו סקריפט bash אשר מממש באופן חלקי את הפקודה head של unix. לסקריפט אסור, כמובן, לקרוא לפקודה head עצמה. הסקריפט יקבל שם קובץ כפרמטר וידפיס את 10 השורות הראשונות ל-stdout.

```
#!/bin/bash
```

```
if [[ $# -ne 1 ]]
```

```
then
```

```
    echo -e "\nUsage: $0 <file-name>\n"
```

```
    exit
```

```
fi
```

```
x=0
```

```
while read line
```

```
do
```

```
    echo $line
```

```
    if [[ $((++x)) -eq 10 ]]
```

```
    then
```

```
        break
```

```
    fi
```

```
done < ${1}
```

[illegible]

המכללה האקדמית תל חי, החוג למדעי המחשב

פקודות bash

- cat – מדפיסה תוכן של קובץ למסך
- head – מדפיסה את 10 השורות הראשונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב -n)
- tail – מדפיסה את 10 השורות האחרונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב -n)
- ls – מדפיסה תוכן ספרייה, או רשימה של קבצים אשר מתאימה לתאור שהעביר המשתמש כארגומנט (למשל: ls *.c תדפיס את שמות הקבצים שמסתיימים ב-c. בספרייה הנוכחית)
- sort – ממיינת לקסיקוגרפית קובץ קלט. ניתן למיין מספרית (-n) ולשמור רק עותק אחד של שורות שחוזרות על עצמן (-u)
- wc – מחזירה את מספר השורות (-l), בתים (-c) או מילים (-w) בקובץ
- read – קוראת שורה מה-standard input ומכניסה אותה למשתנה שמועבר ע"י המשתמש כארגומנט
- find – מחזירה רשימה של כל הקבצים אשר מתאימים לתאור המשתמש (ארגומנט -name) בספרייה מסוימת ואלו שנמצאות תחתיה (ארגומנט ראשון לפקודה)
- grep – מחזירה שורות אשר מכילות string מסויים. ניתן להשתמש ב-regular expressions (-e), ולהחזיר שורות שאינן מכילות את ה-string אותו מחפשים (-v)
- egrep – זהה ל-grep -e
- cut – מחזירה שדות מסויימים בכל שורה (-f) בהתאם ל-delimiter (-d). ניתן גם להחזיר בתים מסויימים בכל שורה (-b)

פעולות על ביטים

- bitwise AND - &
- bitwise OR - |
- bitwise XOR - ^
- right shift - >>
- left shift - <<
- One's complement - ~

פונקציות ב-C

- int putchar(int character, FILE* fp) – כותבת את character ל-fp.
- int getc(FILE* fp) – קוראת תו אחד מתוך fp.
- scanf(const char* format,...) – קוראת מידע מתוך stdin בהתאם ל-format, מאחסנת את המידע בכתובות המשתנים שמסופקים. fscanf(FILE* stream, const char* format,...) – אותו דבר כאשר הקלט נקרא מתוך stream.
- sscanf(const char* s, const char* format,...) – אותו הדבר כאשר הקלט נקרא מתוך s.
- printf(const char* format,...) – כתיבת מידע ערוך ל-stdout בהתאם ל-format והמשתנים שמסופקים.
- fprintf(FILE* stream, const char* format,...) – אותו דבר כאשר הכתיבה נעשית לתוך stream.
- sprintf(const char* s, const char* format,...) – אותו הדבר כאשר הכתיבה נעשית לתוך s.
- void* malloc(size_t size) – מקצה זכרון בגודל size
- void* calloc(size_t num, size_t size) – מקצה num יחידות בגודל size כל אחת ומאפסת את כל הזכרון שהוקצה.
- void* realloc(void* ptr, size_t size) – מקצה size בתים, מוסיפה/מורידה מכמות הזכרון ש-ptr מצביעה עליו (צריך להיות מוקצה דינמית גם כן), עשויה לשנות את הזכרון המוקצה ואז תעתיק את התוכן של ptr לשטח החדש. מחזירה את כתובת הזכרון שהוקצה.
- const char* strstr(const char* str1, const char* str2) – מחזירה את המקום שבו מופיעה str2 ב-str1 (או NULL אם היא לא מופיעה)
- int strcmp(const char* str1, const char* str2) – משווה את str1 ל-str2, מחזירה 0 אם הן זהות, ערך קטן מ-0 אם str1 קטן לקסיקוגרפית מ-str2 או ערך גדול מ-0 אחרת.
- size_t strlen(const char* str) – מחזירה אורך של מחרוזת.